1. A programmer wants to represent different cuboid-shaped objects in a Java program.

Initiate a program using inheritance where a superclass named Rectangle contains the properties length and breadth,
a constructor to initialize them, and a method to calculate the area.

Then create a subclass named Cuboid that inherits from Rectangle and adds an extra property named height.
The Cuboid class should have a constructor that initializes length, breadth, and height using constructor chaining,
and a method to calculate the volume.

In the main method, create an array of three Cuboid objects, initialize them with different values,
and display the length, breadth, height, area, and volume of each cuboid.

2. PayEase is a digital payments app that allows users to pay using different payment methods.

Every payment method performs one common action named processPayment(amount), but the calculation varies depending on the payment type.

MobileBanking adds a 1% service charge to the given amount.
CreditCard adds a flat 20 transaction fee.
Cash charges exactly the given amount without any extra fee.

Create a Java program using a superclass named PaymentMethod and three subclasses named MobileBanking, CreditCard, and Cash.
Each subclass must override the processPayment(amount) method according to its own payment rule.

In the Main method, create/use the payment objects to achieve late binding and display the final payment.

3. Develop a banking application containing a class Account with accountNumber, accountHolder, and balance.

Use a constructor to initialize the account.

Overload deposit() and withdraw() methods to allow depositing or withdrawing
either with an integer amount or an amount with a transaction description.

Demonstrate the system with multiple accounts.

4. Create a superclass LibraryItem containing itemId, title, and author.

Create subclasses Book and Magazine. Use constructors and super to initialize common attributes.

Override calculateFine(int lateDays) to handle books and magazines. Books and magazines have different fine rates.
Implement a display() method to show the information.
Create objects of both subclasses and demonstrate the system.

5. A university wants to develop a Java program to manage information about students who receive scholarships.

Create a class named Person with an attribute name, a constructor for initialization,
and a displayInfo() method to show basic information.

Create a class named Student that extends Person and adds studentId and marks.
Override displayInfo() to display the student's information, ID, and marks.

Then create a class named ScholarshipStudent that extends Student and adds tuitionFee.
Override displayInfo() again to display the complete student information.

Implement a calculateScholarship() method using these rules:
- marks >= 80: 50% of tuition fee
- marks 70-79: 30%
- marks 60-69: 20%
- below 60: no scholarship

In main(), create a ScholarshipStudent object and assign it to a Person reference using upcasting.

Invoke displayInfo() through the parent reference to demonstrate dynamic method dispatch and late binding.
Finally, display the calculated scholarship amount.

6. A university email system has different types of users, such as professors and students.
Each type of user should send emails differently.

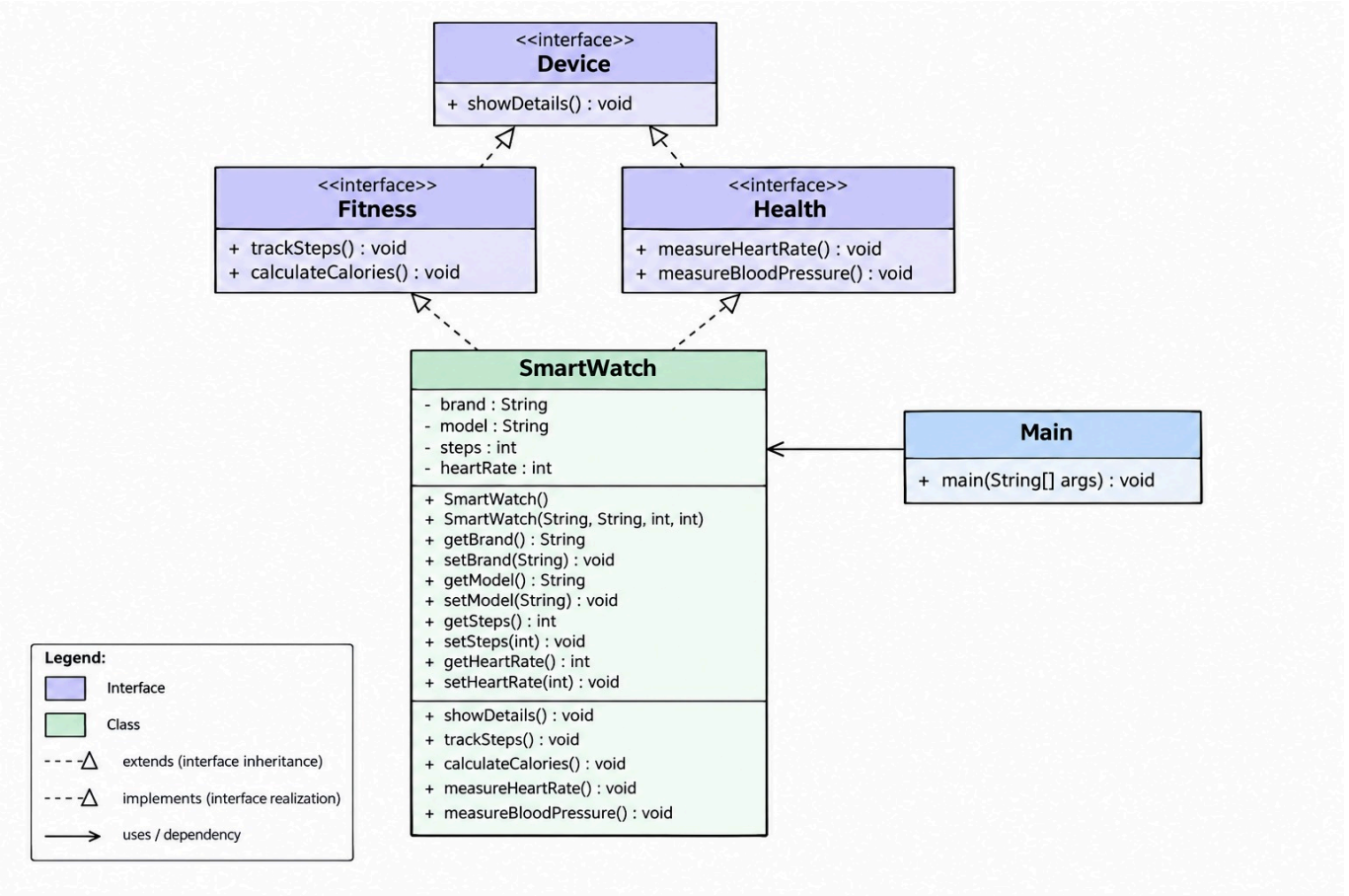
Design a Java program using method overriding and demonstrate dynamic method dispatch when sending an email from a professor account.

7. A food delivery system needs to calculate delivery charges in different ways depending on the information provided.

Using method overloading, design methods to calculate the charge based on:
1. Distance only.
2. Distance and order weight.
3. Distance, order weight, and premium membership status.

Demonstrate the overloaded methods from the main() method.
Also state whether the main() method can be overloaded in Java.

8.



9. Create a superclass Person containing private data members name and age. Implement default and parameterized constructors and setter methods.

Create two subclasses, Employee and BranchManager, using hierarchical inheritance.
The Employee class contains employeeId and salary, while the BranchManager class contains branchName and staffCount.

Use constructors and super() to initialize common attributes.
Override the show() method in both subclasses to display all information.

In Main, create arrays of two Employee objects and two BranchManager objects.
Use loops to display the information of all employees and branch managers.

SEC-02

1. (a)

Complete the following Java program by filling in the blanks based on the given hints. Ensure that the completed program correctly displays the online customer's name, ID, and item count.

```
class Customer {
    ____ String customerName;
    // Accessible from any class
    ____ int customerId;
    // Accessible only within Customer

    Customer(String name, int id) {
        this.customerName = ____;
        this.customerId = id;
    }
}

    class ____ {
        private int itemCount;

        Order(int count) {
        this.itemCount = ____;
    }

    void display() {
        System.out.println(customerName +    " "
                + customerId + " " + ____);
    }
}

    void showCustomer() {
        ____ item = new ____(4);
        item.display();
    }
}

public class Main {
    public static void main(String[] args) {
        Store obj = new Store("Arif", 701);
        obj.____();
    }
}
```

1. (b)

An online store organization wants to evaluate the performance of its customers. Create a Java program using two different packages, `store` and `management`.

In the `store` package, create a `Customer` class with three private data members: `customerName`, `customerId`, and `loyaltyScore`. Use appropriate getter and setter methods to access and modify these data members. The `loyaltyScore` must be validated to accept only values between 0 and 100.

Implement a method `calculateLoyaltyTier()` to classify the score as Platinum (80–100), Gold (60–79), Silver (40–59), or Bronze (below 40).

In the `management` package, create a `CustomerManagement` class containing the `main()` method, create objects for three customers, assign their information using setter methods, and display each customer's name, ID, score, and category.

SEC-03

1. (a)

Complete the following Java program by filling in the blanks based on the given hints. Ensure that the completed program correctly displays the sports academy's name, ID, and training hours.

```
class Athlete {
    ____ String athleteName;
    // Accessible from any class

    ____ int athleteId;
    // Accessible only within Athlete

    Athlete(String name, int id) {
        this.athleteName = ____;
        this.athleteId = id;
    }

    class ____ {
        private int trainingHours;

        Training(int hours) {
            this.trainingHours = ____;
        }

        void display() {
            System.out.println(athleteName + " "
                    + athleteId + " " + ____);
        }
    }

    void showAthlete() {
        ____ item = new ____(20);
        item.display();
    }
}

public class Main {
    public static void main(String[] args) {
        Athlete obj = new Athlete("Sakib", 501);
        obj.____();
    }
}
```

1. (b)

A sports academy wants to evaluate the training performance of its athletes. Create a Java program using two different packages, `academy` and `management`.

In the `academy` package, create an `Athlete` class with three private data members: `athleteName`, `athleteId`, and `performanceScore`. Use appropriate getter and setter methods to access and modify these data members. The `performanceScore` must be validated to accept only values between 0 and 100.

Implement a method `calculatePerformanceLevel()` to classify the result as Elite (80–100), Advanced (60–79), Intermediate (40–59), or Beginner (below 40).

In the `management` package, create an `AthleteManagement` class containing the `main()` method. Create objects for three athletes, assign their information using setter methods, and display each athlete's name, ID, score, and performance level.